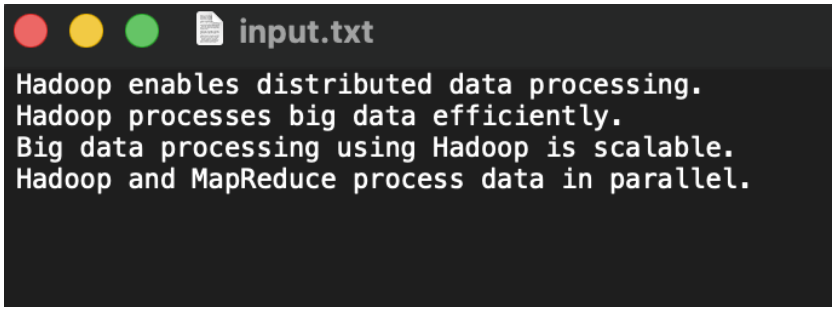
Assignment 1: MapReduce and Apache Spark

Name: Sushantak Parashar Jha
Roll No: M25CSA035
Program: M.Tech in AI
Course: Machine Learning With Big Data(CSL7110)
GitHub Repository and Pages Link: ***Link***

Question 1 - Execution of WordCount Example

In this experiment, a single-node Hadoop cluster was tested by executing the standard WordCount MapReduce example. A sample text file was created locally and uploaded to HDFS, after which the WordCount program was executed using the Hadoop examples JAR file.

During execution, the mapper generated intermediate key–value pairs of the form (word, 1) for each word in the input text, and the reducer aggregated these values to compute the total frequency of each word. Upon successful completion, Hadoop created an output directory containing the _SUCCESS file and the part-r-00000 file with the final results, confirming the correct functioning of HDFS and the MapReduce processing framework.



A custom input text file containing sample text with repeated words was created to demonstrate the working of the WordCount MapReduce program.

```
sushantakparasharjha@Sushantak ~ % hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar wordcount \
/user/$USER/input /user/$USER/output

2026-02-06 00:14:50,670 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-02-06 00:14:50,868 INFO impl.MetricsConfig: Loaded properties from hadoop-metrics2.properties
2026-02-06 00:14:50,895 INFO impl.MetricsSystemImpl: Scheduled Metric snapshot period at 10 second(s).
2026-02-06 00:14:50,895 INFO impl.MetricsSystemImpl: JobTracker metrics system started
2026-02-06 00:14:51,007 INFO input.FileInputFormat: Total input files to process : 1
2026-02-06 00:14:51,036 INFO mapreduce.JobSubmitter: number of splits:1
2026-02-06 00:14:51,095 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_local687043458_0001
2026-02-06 00:14:51,095 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-02-06 00:14:51,146 INFO mapreduce.Job: The url to track the job: http://localhost:8080/
2026-02-06 00:14:51,147 INFO mapreduce.Job: Running job: job_local687043458_0001
2026-02-06 00:14:51,147 INFO mapred.LocalJobRunner: OutputCommitter set in config null
2026-02-06 00:14:51,150 INFO output.PathOutputCommitterFactory: No output committer factory defined, defaulting to FileOutputCommitterFactory
2026-02-06 00:14:51,150 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
2026-02-06 00:14:51,150 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup _temporary folders under output directory:false, ignore cleanup failures: false
2026-02-06 00:14:51,150 INFO mapred.LocalJobRunner: OutputCommitter is org.apache.hadoop.mapreduce.lib.output.FileOutputCommitter
2026-02-06 00:14:51,177 INFO mapred.LocalJobRunner: Waiting for map tasks
2026-02-06 00:14:51,177 INFO mapred.LocalJobRunner: Starting task: attempt_local687043458_0001_m_000000_0
2026-02-06 00:14:51,184 INFO output.PathOutputCommitterFactory: No output committer factory defined, defaulting to FileOutputCommitterFactory
2026-02-06 00:14:51,184 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
2026-02-06 00:14:51,184 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup _temporary folders under output directory:false, ignore cleanup failures: false
2026-02-06 00:14:51,188 INFO util.ProcfsBasedProcessTree: ProcfsBasedProcessTree currently is supported only on Linux.
2026-02-06 00:14:51,188 INFO mapred.Task:  Using ResourceCalculatorProcessTree : null
2026-02-06 00:14:51,190 INFO mapred.MapTask: Processing split: hdfs://localhost:9000/user/sushantakparasharjha/input/input.txt:0+177
2026-02-06 00:14:51,202 INFO mapred.MapTask: (EQUATOR) 0 kvi 26214396(104857584)
2026-02-06 00:14:51,202 INFO mapred.MapTask: mapreduce.task.io.sort.mb: 100
2026-02-06 00:14:51,202 INFO mapred.MapTask: soft limit at 83886080
2026-02-06 00:14:51,202 INFO mapred.MapTask: bufstart = 0; bufvoid = 104857600
2026-02-06 00:14:51,202 INFO mapred.MapTask: kvstart = 26214396; length = 6553600
2026-02-06 00:14:51,203 INFO mapred.MapTask: Map output collector class = org.apache.hadoop.mapred.MapTask$MapOutputBuffer
2026-02-06 00:14:51,240 INFO mapred.LocalJobRunner:
2026-02-06 00:14:51,241 INFO mapred.MapTask: Starting flush of map output
2026-02-06 00:14:51,241 INFO mapred.MapTask: Spilling map output
2026-02-06 00:14:51,241 INFO mapred.MapTask: bufstart = 0; bufend = 272; bufvoid = 104857600
2026-02-06 00:14:51,241 INFO mapred.MapTask: kvstart = 26214396(104857584); kvend = 26214304(104857216); length = 93/6553600
2026-02-06 00:14:51,253 INFO mapred.MapTask: Finished spill 0
2026-02-06 00:14:51,259 INFO mapred.Task: Task:attempt_local687043458_0001_m_000000_0 is done. And is in the process of committing
2026-02-06 00:14:51,260 INFO mapred.LocalJobRunner: map
2026-02-06 00:14:51,260 INFO mapred.Task: Task 'attempt_local687043458_0001_m_000000_0' done.
2026-02-06 00:14:51,262 INFO mapred.Task: Final Counters for attempt_local687043458_0001_m_000000_0: Counters: 24
        File System Counters
                FILE: Number of bytes read=281531
                FILE: Number of bytes written=942311
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=177
                HDFS: Number of bytes written=0
                HDFS: Number of read operations=5
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=1
                HDFS: Number of bytes read erasure-coded=0
        Map-Reduce Framework
                Map input records=5
                Map output records=24
                Map output bytes=272
                Map output materialized bytes=254
                Input split bytes=128
                Combine input records=24
                Combine output records=18
```

Executing the WordCount MapReduce job

```
sushantakparasharjha@Sushantak ~ % hdfs dfs -ls /user/sushantakparasharjha/output

2026-02-06 00:15:50,076 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 2 items
-rw-r--r--   1 sushantakparasharjha supergroup          0 2026-02-06 00:14 /user/sushantakparasharjha/output/_SUCCESS
-rw-r--r--   1 sushantakparasharjha supergroup        176 2026-02-06 00:14 /user/sushantakparasharjha/output/part-r-00000
```

Listing the HDFS output directory after job completion

```
sushantakparasharjha@Sushantak ~ % hdfs dfs -cat /user/$USER/output/part-r-00000

2026-02-06 00:16:34,442 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Big     1
Hadoop  4
MapReduce       1
and     1
big     1
data    4
distributed     1
efficiently.    1
enables 1
in      1
is      1
parallel.       1
process 1
processes       1
processing      1
processing.     1
scalable.       1
using   1
sushantakparasharjha@Sushantak ~ %
```

Displaying the reducer output (word frequencies)

Question 2: Map Phase Key–Value Pairs

In the Map phase, Hadoop reads each line of the input file and provides it to the mapper as a key–value pair where the key represents the byte offset of the line and the value represents the text content of the line. For the given lyrics example, the mapper receives input pairs such as:

(0, "We're up all night till the sun")
(31, "We're up all night to get some")
(63, "We're up all night for good fun")
(95, "We're up all night to get lucky")

The mapper processes each line, tokenizes it into words, and emits intermediate key–value pairs of the form (word, 1), for example:

("We're", 1), ("up", 1), ("all", 1), ("night", 1), ("till", 1), ("the", 1), ("sun", 1)
("We're", 1), ("up", 1), ("all", 1), ("night", 1), ("to", 1), ("get", 1), ("some", 1)
("We're", 1), ("up", 1), ("all", 1), ("night", 1), ("for", 1), ("good", 1), ("fun", 1)
("We're", 1), ("up", 1), ("all", 1), ("night", 1), ("to", 1), ("get", 1), ("lucky", 1)

Map Phase Types
Input Key Type: **LongWritable**
Input Value Type: **Text**
Output Key Type: **Text**
Output Value Type: **IntWritable**

Question 3: Reduce Phase Key–Value Pairs

After the Map phase, Hadoop performs the shuffle and sort operation, where intermediate key–value pairs having the same key are grouped together. The reducer receives grouped inputs such as:

("we're", [1, 1, 1, 1])
("up", [1, 1, 1, 1])
("all", [1, 1, 1, 1])
("night", [1, 1, 1, 1])
("to", [1, 1])
("get", [1, 1])
("till", [1])

("sun", [1])
("good", [1])
("fun", [1])
("lucky", [1])

The reducer sums the values associated with each word to generate the final word counts, producing outputs such as:

("we’re", 4), ("up", 4), ("all", 4), ("night", 4), ("to", 2), ("get", 2), ("till", 1), ("sun", 1), ("good", 1), ("fun", 1), ("lucky", 1)

Reduce Phase Types
Input Key Type: **Text**
Input Value Type: **Iterable<IntWritable>**
Output Key Type: **Text**
Output Value Type: **IntWritable**

Question 4 — Filling the Placeholders in WordCount.java

From the Hadoop MapReduce tutorial WordCount program, the placeholders in the Map and Reduce function definitions are replaced using the data types identified in Question 2.

Map function definition

```
Java

public void map(LongWritable key, Text value, Context context)
    throws IOException, InterruptedException {
  ...
}
```

Reduce function definition

```
Java

public void reduce(Text key, Iterable<IntWritable> values, Context context)
    throws IOException, InterruptedException {
  ...
}
```

Job output key and value classes

Java

```
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(IntWritable.class);
```

Explanation:

The Mapper receives input key–value pairs of type (LongWritable, Text) and emits intermediate (Text, IntWritable) pairs. The Reducer processes grouped values of type Iterable<IntWritable> and produces the final output of type (Text, IntWritable).

Question 5 — Writing the map() Function

The map() function removes punctuation using String.replaceAll(), splits the input line into words using StringTokenizer, and emits (word, 1) pairs.

Java

```
public void map(LongWritable key, Text value, Context context)
    throws IOException, InterruptedException {

  String line = value.toString().replaceAll("[^a-zA-Z ]", " ");
  StringTokenizer itr = new StringTokenizer(line);

  while (itr.hasMoreTokens()) {
    Text word = new Text(itr.nextToken().toLowerCase());
    context.write(word, new IntWritable(1));
  }
}
```

Explanation:

The map() function processes each input line, removes punctuation characters, tokenizes the cleaned line into words, and emits intermediate key–value pairs (word, 1) for each word. This prepares the data for aggregation in the Reduce phase.

Question 6 — Writing the reduce() Function

The reduce() function receives each word along with a collection of integer values generated by the Mapper. It sums these values to compute the total number of occurrences of each word and outputs the final result.

reduce() Function

```
Java

public void reduce(Text key, Iterable<IntWritable> values,
                   Context context)
    throws IOException, InterruptedException {

  int sum = 0;

  // Sum all values associated with the word
  for (IntWritable val : values) {
     sum += val.get();
  }

  // Emit final word count
  context.write(key, new IntWritable(sum));
}
```

Explanation:

The reduce() function processes grouped intermediate key–value pairs produced during the Map phase. For each word (key), it iterates through the list of associated integer values (Iterable<IntWritable>), calculates their total, and outputs the final word frequency as (word, total_count).

Question 7 — Compile and Run the WordCount Program

After implementing the Mapper and Reducer functions, the WordCount.java program was compiled using the Hadoop classpath and packaged into a JAR file. The dataset 200.txt was uploaded to the Hadoop Distributed File System (HDFS), and the custom WordCount MapReduce job was executed. Since Hadoop does not allow overwriting an existing output directory, the previous output folder was removed before running the job again. After successful execution, the reducer output files were merged into a single file (result.txt), which contains the final word frequency counts generated by the program.

```
sushantakparasharjha@Sushantak wordcount_project % javac -classpath `hadoop classpath` WordCount.java
```

Compilation of the WordCount.java program using the Hadoop classpath.

```
sushantakparasharjha@Sushantak wordcount_project % jar cf WordCount.jar WordCount*.class
```

Creation of the executable WordCount.jar file containing the compiled classes.

```
sushantakparasharjha@Sushantak wordcount_project % hadoop fs -copyFromLocal 200.txt

2026-02-06 12:19:38,979 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform...
copyFromLocal: `200.txt': File exists
```

Copying the dataset file 200.txt from the local filesystem to the Hadoop Distributed File System (HDFS) using the hadoop fs -copyFromLocal command.

```
sushantakparasharjha@Sushantak wordcount_project % hadoop jar WordCount.jar WordCount 200.txt output

2026-02-06 12:04:59,109 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java
2026-02-06 12:04:59,310 INFO impl.MetricsConfig: Loaded properties from hadoop-metrics2.properties
2026-02-06 12:04:59,339 INFO impl.MetricsSystemImpl: Scheduled Metric snapshot period at 10 second(s).
2026-02-06 12:04:59,339 INFO impl.MetricsSystemImpl: JobTracker metrics system started
2026-02-06 12:04:59,389 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool
2026-02-06 12:04:59,438 INFO input.FileInputFormat: Total input files to process : 1
2026-02-06 12:04:59,481 INFO mapreduce.JobSubmitter: number of splits:1
2026-02-06 12:04:59,538 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_local431806245_0001
2026-02-06 12:04:59,538 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-02-06 12:04:59,585 INFO mapreduce.Job: The url to track the job: http://localhost:8080/
2026-02-06 12:04:59,585 INFO mapreduce.Job: Running job: job_local431806245_0001
2026-02-06 12:04:59,586 INFO mapred.LocalJobRunner: OutputCommitter set in config null
2026-02-06 12:04:59,588 INFO output.PathOutputCommitterFactory: No output committer factory defined, defaulting to FileOutputComm
2026-02-06 12:04:59,588 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
2026-02-06 12:04:59,588 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup _temporary folders under output directo
2026-02-06 12:04:59,588 INFO mapred.LocalJobRunner: OutputCommitter is org.apache.hadoop.mapreduce.lib.output.FileOutputCommitte
2026-02-06 12:04:59,613 INFO mapred.LocalJobRunner: Waiting for map tasks
2026-02-06 12:04:59,613 INFO mapred.LocalJobRunner: Starting task: attempt_local431806245_0001_m_000000_0
2026-02-06 12:04:59,620 INFO output.PathOutputCommitterFactory: No output committer factory defined, defaulting to FileOutputComm
2026-02-06 12:04:59,620 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
2026-02-06 12:04:59,620 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup _temporary folders under output directo
2026-02-06 12:04:59,623 INFO util.ProcfsBasedProcessTree: ProcfsBasedProcessTree currently is supported only on Linux.
2026-02-06 12:04:59,624 INFO mapred.Task:  Using ResourceCalculatorProcessTree : null
2026-02-06 12:04:59,625 INFO mapred.MapTask: Processing split: hdfs://localhost:9000/user/sushantakparasharjha/200.txt:0+8312639
2026-02-06 12:04:59,643 INFO mapred.MapTask: (EQUATOR) 0 kvi 26214396(104857584)
2026-02-06 12:04:59,643 INFO mapred.MapTask: mapreduce.task.io.sort.mb: 100
2026-02-06 12:04:59,643 INFO mapred.MapTask: soft limit at 83886080
2026-02-06 12:04:59,643 INFO mapred.MapTask: bufstart = 0; bufvoid = 104857600
2026-02-06 12:04:59,643 INFO mapred.MapTask: kvstart = 26214396; length = 6553600
2026-02-06 12:04:59,644 INFO mapred.MapTask: Map output collector class = org.apache.hadoop.mapred.MapTask$MapOutputBuffer
2026-02-06 12:05:00,031 INFO mapred.LocalJobRunner:
2026-02-06 12:05:00,032 INFO mapred.MapTask: Starting flush of map output
2026-02-06 12:05:00,032 INFO mapred.MapTask: Spilling map output
2026-02-06 12:05:00,032 INFO mapred.MapTask: bufstart = 0; bufend = 12867576; bufvoid = 104857600
2026-02-06 12:05:00,032 INFO mapred.MapTask: kvstart = 26214396(104857584); kvend = 20939320(83757280); length = 5275077/6553600
2026-02-06 12:05:00,404 INFO mapred.MapTask: Finished spill 0
2026-02-06 12:05:00,412 INFO mapred.Task: Task:attempt_local431806245_0001_m_000000_0 is done. And is in the process of committi
2026-02-06 12:05:00,413 INFO mapred.LocalJobRunner: map
2026-02-06 12:05:00,413 INFO mapred.Task: Task 'attempt_local431806245_0001_m_000000_0' done.
```

Execution of the custom WordCount MapReduce job on the dataset stored in HDFS.

```
sushantakparasharjha@Sushantak wordcount_project % hdfs dfs -getmerge output result.txt
cat result.txt

2026-02-06 12:07:18,223 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform...
a       26012
aa      10
aaa     1
aabborgt        1
aachen  7
aachens 1
aad     1
aadms   1
aagesen 2
aahmes  2
aak     1
aal     1
aalborg 4
aalen   2
aalesund        4
aali    2
aalst   1
aar     13
aarau   12
aarberg 1
aarburg 2
aard    6
aardrjjko       1
aare    1
aargau  10
aargauer        1
aargaus 1
aarhus  5
aaron   19
aaronite        1
aaronites       1
aarssen 1
aarssens        2
aartsen 1
```

Reducer output merged into a single file showing the final word frequency counts.

Question 8 - Copying data to HDFS

```
sushantakparasharjha@Sushantak wordcount_project % hadoop fs -copyFromLocal 200.txt

2026-02-06 12:19:38,979 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform...
copyFromLocal: `200.txt': File exists
```

Copying the dataset file 200.txt from the local filesystem to the Hadoop Distributed File System (HDFS) using the hadoop fs -copyFromLocal command.

```
sushantakparasharjha@Sushantak wordcount_project % hadoop fs -ls

2026-02-06 12:22:02,937 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform...
using builtin-java classes where applicable
Found 3 items
-rw-r--r--   1 sushantakparasharjha supergroup    8312639 2026-02-06 12:02 200.txt
drwxr-xr-x   - sushantakparasharjha supergroup          0 2026-02-06 00:14 input
drwxr-xr-x   - sushantakparasharjha supergroup          0 2026-02-06 12:05 output
```

Listing the contents of the HDFS directory using the hadoop fs -ls command, showing the uploaded file along with the existing directories and their replication information.

The hadoop fs -ls command lists the files and directories stored in the Hadoop Distributed File System (HDFS). In the displayed output, the first entry represents the file 200.txt, while the other entries (input and output) represent directories.

The second column in the listing shows the replication factor for files. For 200.txt, the replication factor is 1, meaning that one copy of the file is stored in HDFS.

Why don't we have a replication factor for directories?

-> Directories do not display a replication factor because they do not store actual data blocks; instead, they contain metadata describing the files stored inside them.

The replication factor determines the reliability and performance of the system. Increasing the replication factor improves fault tolerance and read performance because multiple copies of the file are available across different nodes, but it increases storage consumption and write overhead. Decreasing the replication factor reduces storage usage but lowers fault tolerance in case of node failures.

Question 9 - Total execution time of the job

To measure the execution time of the WordCount MapReduce program, timing statements were added in the main() method using System.currentTimeMillis(). The start time was recorded before submitting the job and the end time after job completion. The difference between these values was printed to the console, representing the total execution time of the MapReduce job.

```
sushantakparasharjha@Sushantak wordcount_project % hadoop jar WordCount.jar WordCount 200.txt output

2026-02-06 16:08:23,108 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform...
using builtin-java classes where applicable
2026-02-06 16:08:23,331 INFO impl.MetricsConfig: Loaded properties from hadoop-metrics2.properties
2026-02-06 16:08:23,362 INFO impl.MetricsSystemImpl: Scheduled Metric snapshot period at 10 second(s).
2026-02-06 16:08:23,363 INFO impl.MetricsSystemImpl: JobTracker metrics system started
2026-02-06 16:08:23,423 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed.
Implement the Tool interface and execute your application with ToolRunner to remedy this.
```

```
        File Input Format Counters
                Bytes Read=8312639
        File Output Format Counters
                Bytes Written=675870
Execution time: 2360 ms
```

Executing the WordCount MapReduce job with execution time measurement enabled.

```
sushantakparasharjha@Sushantak wordcount_project % hadoop jar WordCount.jar WordCount 200.txt output

2026-02-06 16:20:49,826 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your pla
using builtin-java classes where applicable
2026-02-06 16:20:50,051 INFO impl.MetricsConfig: Loaded properties from hadoop-metrics2.properties
2026-02-06 16:20:50,084 INFO impl.MetricsSystemImpl: Scheduled Metric snapshot period at 10 second(s).
2026-02-06 16:20:50,084 INFO impl.MetricsSystemImpl: JobTracker metrics system started
2026-02-06 16:20:50,142 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not pe
Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-02-06 16:20:50,190 INFO input.FileInputFormat: Total input files to process : 1
```

```
        File Input Format Counters
                Bytes Read=8312639
        File Output Format Counters
                Bytes Written=675870
Execution time: 2338 ms
```

Execution time of the WordCount MapReduce job after modifying the input split size configuration parameter.

How does changing the value of that parameter impact performance? Why?

-> The parameter mapreduce.input.fileinputformat.split.maxsize determines the maximum size of each input split, which directly affects the number of mapper tasks created during job execution. Increasing the split size lowers the number of mapper tasks, which lowers the overhead associated with task scheduling but also decreases parallelism, which could cause processing in distributed environments to lag. Although more mapper tasks are created and managed when the split size is decreased, this improves parallelism and enables the processing of more data at once. However, excessive splitting can result in higher overhead. Thus, choosing the ideal split size that strikes a balance between parallelism and task management overhead determines how well a MapReduce job performs.

Question 10 - Book Metadata Extraction and Analysis

The text file reader was used to load the Project Gutenberg book collection into Apache Spark. Every book's line was saved as a row in a DataFrame, along with the file name that went with it. The metadata fields like title, release date, language, and encoding were then extracted from the book headers using regular expressions.

To facilitate additional analysis, the extracted metadata was arranged into a structured DataFrame. Additional exploratory calculations, such as the average title length, the most prevalent language in the dataset, and the number of books published annually, were carried out using this metadata. The dataset was ready for later text processing and similarity analysis tasks thanks to this preprocessing step.

```
>>> from pyspark.sql.functions import *
...
... # Load books dataset
... books_df = spark.read.text("file:///Users/sushantakparasharjha/D184MB/") \
...     .withColumnRenamed("value","text") \
...     .withColumn("file_name", input_file_name())
...
... books_df.show(5, False)
...
```

```
+------------------------------------------------------------------------+----------------------------------------------+
|text                                                                    |file_name                                     |
+------------------------------------------------------------------------+----------------------------------------------+
|The Project Gutenberg EBook of The Project Gutenberg Gutenberg          |file:///Users/sushantakparasharjha/D184MB/200.txt|
|Encyclopedia, Vol 1, by Project Gutenberg                               |file:///Users/sushantakparasharjha/D184MB/200.txt|
|                                                                        |file:///Users/sushantakparasharjha/D184MB/200.txt|
|This eBook is for the use of anyone anywhere at no cost and with        |file:///Users/sushantakparasharjha/D184MB/200.txt|
|almost no restrictions whatsoever.  You may copy it, give it away or|file:///Users/sushantakparasharjha/D184MB/200.txt|
+------------------------------------------------------------------------+----------------------------------------------+
only showing top 5 rows
```

Displaying sample records after loading the Project Gutenberg books dataset into a Spark DataFrame

```
... # Metadata extraction
... metadata_df = books_df \
...     .filter(
...         lower(col("text")).startswith("title:") |
...         lower(col("text")).startswith("release date:") |
...         lower(col("text")).startswith("language:") |
...         lower(col("text")).startswith("encoding:")
...     ) \
...     .groupBy("file_name") \
...     .agg(
...         max(when(lower(col("text")).startswith("title:"), col("text"))).alias("title"),
...         max(when(lower(col("text")).startswith("release date:"), col("text"))).alias("release_date"),
...         max(when(lower(col("text")).startswith("language:"), col("text"))).alias("language"),
...         max(when(lower(col("text")).startswith("encoding:"), col("text"))).alias("encoding")
...     )
...
... metadata_df.show(5, False)
...
```

```
+----------------------------------------+-----------------------------------------------------+-----------------------------+----------+--------+
|file_name                               |title                                                |release_date                 |language  |encoding|
+----------------------------------------+-----------------------------------------------------+-----------------------------+----------+--------+
|file:///Users/sushantakparasharjha/D184MB/10.txt |Title: The King James Bible                          |Release Date: March 2, 2011 [EBook #10]|Language: English|NULL    |
|file:///Users/sushantakparasharjha/D184MB/101.txt|Title: Hacker Crackdown                              |Release Date: January, 1994  |Language: English|NULL    |
|file:///Users/sushantakparasharjha/D184MB/102.txt|Title: The Tragedy of Pudd'nhead Wilson              |Release Date: January, 1994  |Language: English|NULL    |
|file:///Users/sushantakparasharjha/D184MB/103.txt|Title: Around the World in 80 Days                   |Release Date: May 15, 2008 [EBook #103]|Language: English|NULL    |
|file:///Users/sushantakparasharjha/D184MB/104.txt|Title: Franklin Delano Roosevelt's First Inaugural Address|Release Date: May 14, 2008 [EBook #104]|Language: English|NULL    |
+----------------------------------------+-----------------------------------------------------+-----------------------------+----------+--------+
only showing top 5 rows
```

Using regular expressions, the metadata fields (title, release date, language, and encoding) are extracted from the book headers and the metadata_df is displayed.

```
scala> val books_per_year = metadata_df
     |   .withColumn("year", regexp_extract(col("release_date"), "(\\d{4})", 1))
     |   .filter(col("year") =!= "")
     |   .groupBy("year")
     |   .count()
     |   .orderBy("year")
     |
     | books_per_year.show()
+----+-----+
|year|count|
+----+-----+
|1975|    1|
|1978|    1|
|1979|    1|
|1991|    7|
|1992|   19|
|1993|   13|
|1994|   17|
|1995|   60|
|1996|   53|
|2002|    1|
|2004|    7|
|2005|    4|
|2006|   42|
|2007|   13|
|2008|  154|
|2009|    1|
|2010|    9|
|2011|    1|
|2012|    2|
|2013|    1|
+----+-----+
only showing top 20 rows
val books_per_year: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [year: string, count: bigint]
```

Calculating the number of books released each year.

```
... # Books released each year
... books_per_year = metadata_df \
...     .withColumn("year", regexp_extract(col("release_date"), "(\\d{4})", 1)) \
...     .groupBy("year") \
...     .count()
...
... books_per_year.show()
```

```
+----+-----+
|year|count|
+----+-----+
|2012|    2|
|2013|    1|
|2005|    4|
|NULL|    2|
|1978|    1|
|2002|    1|
|2009|    1|
|1995|   60|
|2006|   42|
|2004|    7|
|2011|    1|
|1992|   19|
|2008|  154|
|1994|   17|
|2007|   13|
|1996|   53|
|1979|    1|
|2015|    1|
|1993|   13|
|    |    1|
+----+-----+
only showing top 20 rows
```

```
... # Most common language
... metadata_df.groupBy("language") \
...     .count() \
...     .orderBy(desc("count")) \
...     .show(5)
```

```
+--------------------+-----+
|            language|count|
+--------------------+-----+
|   Language: English|  403|
|     Language: Latin|    6|
|Language: total p...|    1|
|language: and thu...|    1|
+--------------------+-----+
```

Finding the most common language in the dataset.

```
... # Average title length
... metadata_df \
...     .withColumn("title_length", length(col("title"))) \
...     .select(avg("title_length")) \
...     .show()
```

```
+------------------+
| avg(title_length)|
+------------------+
|29.829268292682926|
+------------------+
```

Calculating the average length of book titles (in characters).

Explain the regular expressions you used to extract the title, release date, language, and encoding. Discuss any challenges or limitations in using regular expressions for this task.

-> Regular expressions were used to identify metadata lines in the text that begin with specific keywords such as “Title:”, “Release Date:”, “Language:”, and “Encoding:”. Pattern matching was performed using expressions that check whether a line starts with these labels, for example ^Title: or ^Release Date:. For extracting the year from the release date, the regular expression (\\d{4}) was used to capture the four-digit year present in the release date string.

One challenge in using regular expressions for metadata extraction is that the formatting of Project Gutenberg files is not always consistent. Some books may use slightly different label formats, contain additional spaces, or place metadata in different sections of the text.As a result, some metadata values might be absent or improperly recorded. Furthermore, not all metadata fields may be present in all files, which could result in null or incomplete records in the DataFrame that is produced.

What are some potential issues with the extracted metadata (e.g., inconsistencies, missing values)? How would you handle these issues in a real-world scenario?

-> Additional data-cleaning procedures, like normalising text formatting, utilising multiple pattern variations to capture various label styles, filling in missing values with default placeholders, and conducting validation checks to make sure extracted metadata is complete and consistent before further analysis, can address these problems in real-world scenarios.

Question 11 - TF-IDF and Book Similarity

```
>>> from pyspark.sql.functions import input_file_name, regexp_replace, lower, col, desc
... from pyspark.ml.feature import Tokenizer, StopWordsRemover, CountVectorizer, IDF, Normalizer
... from pyspark.sql.types import DoubleType
... from pyspark.sql.functions import udf
...
... # Load books
... books_df = spark.read.text("file:///Users/sushantakparasharjha/D184MB/") \
...     .withColumnRenamed("value", "text") \
...     .withColumn("file_name", input_file_name())
...
... books_df.show(5, False)
...
...
... # Clean text
... clean_df = books_df.withColumn(
...     "clean_text",
...     lower(regexp_replace(col("text"), "[^a-zA-Z\\s]", " "))
... )
...
... clean_df.select("clean_text").show(5, False)
...
```

```
+------------------------------------------------------------------------+----------------------------------------------+
|text                                                                    |file_name                                     |
+------------------------------------------------------------------------+----------------------------------------------+
|The Project Gutenberg EBook of The Project Gutenberg Gutenberg          |file:///Users/sushantakparasharjha/D184MB/200.txt|
|Encyclopedia, Vol 1, by Project Gutenberg                               |file:///Users/sushantakparasharjha/D184MB/200.txt|
|                                                                        |file:///Users/sushantakparasharjha/D184MB/200.txt|
|This eBook is for the use of anyone anywhere at no cost and with        |file:///Users/sushantakparasharjha/D184MB/200.txt|
|almost no restrictions whatsoever.  You may copy it, give it away or|file:///Users/sushantakparasharjha/D184MB/200.txt|
+------------------------------------------------------------------------+----------------------------------------------+
only showing top 5 rows
+------------------------------------------------------------------------+
|clean_text                                                              |
+------------------------------------------------------------------------+
|the project gutenberg ebook of the project gutenberg gutenberg          |
|encyclopedia  vol    by project gutenberg                               |
|                                                                        |
|this ebook is for the use of anyone anywhere at no cost and with        |
|almost no restrictions whatsoever   you may copy it  give it away or|
+------------------------------------------------------------------------+
only showing top 5 rows
```

Removing punctuation and changing the book's text to lowercase will clean it up.

```
... 
... # Tokenize words
... tokenizer = Tokenizer(inputCol="clean_text", outputCol="words")
... tokenized_df = tokenizer.transform(clean_df)
... 
... tokenized_df.select("words").show(5, False)
```

```
+------------------------------------------------------------------------------------------------------------+
|words                                                                                                       |
+------------------------------------------------------------------------------------------------------------+
|[the, project, gutenberg, ebook, of, the, project, gutenberg, gutenberg]                                    |
|[encyclopedia, , vol, , , , by, project, gutenberg]                                                         |
|[]                                                                                                          |
|[this, ebook, is, for, the, use, of, anyone, anywhere, at, no, cost, and, with]                             |
|[almost, no, restrictions, whatsoever, , , you, may, copy, it, , give, it, away, or]                        |
+------------------------------------------------------------------------------------------------------------+
only showing top 5 rows
```

Using Spark Tokeniser to tokenise the cleaned text into distinct words.

```
...
... # Remove stopwords
... remover = StopWordsRemover(inputCol="words", outputCol="filtered_words")
... filtered_df = remover.transform(tokenized_df)
...
... filtered_df.select("filtered_words").show(5, False)
...
```

```
+--------------------------------------------------------------------+
|filtered_words                                                      |
+--------------------------------------------------------------------+
|[project, gutenberg, ebook, project, gutenberg, gutenberg]          |
|[encyclopedia, , vol, , , , project, gutenberg]                     |
|[]                                                                  |
|[ebook, use, anyone, anywhere, cost]                                |
|[almost, restrictions, whatsoever, , , may, copy, , give, away]|
+--------------------------------------------------------------------+
only showing top 5 rows
```

To get the data ready for TF-IDF computation, common stopwords are eliminated from the tokenised text.

```
... # TF
... cv = CountVectorizer(inputCol="filtered_words", outputCol="tf_features")
... cv_model = cv.fit(filtered_df)
... tf_df = cv_model.transform(filtered_df)
...
... tf_df.select("tf_features").show(5, False)
```

```
+-----------------------------------------------------------------------+
|tf_features                                                            |
+-----------------------------------------------------------------------+
|(210917,[11,13,517],[3.0,2.0,1.0])                                     |
|(210917,[0,11,13,4555,20603],[4.0,1.0,1.0,1.0,1.0])                    |
|(210917,[0],[1.0])                                                     |
|(210917,[110,517,676,852,1425],[1.0,1.0,1.0,1.0,1.0])                 |
|(210917,[0,15,41,80,159,363,1970,2781],[3.0,1.0,1.0,1.0,1.0,1.0,1.0,1.0])|
+-----------------------------------------------------------------------+
only showing top 5 rows
26/02/08 00:47:02 WARN DAGScheduler: Broadcasting large task binary with size 3.3 MiB
```

Using the trained CountVectorizer model to create term frequency feature vectors for every book.

The CountVectorizer transformation, which turns each book's filtered words into a numerical feature vector that shows the frequency of each term in the text, was used to calculate the Term Frequency (TF) values.

```
... # IDF
... idf = IDF(inputCol="tf_features", outputCol="tfidf_features")
... idf_model = idf.fit(tf_df)
... tfidf_df = idf_model.transform(tf_df)
...
... tfidf_df.select("tfidf_features").show(5, False)
...
```

```
+--------------------------------------------------------------------------------+
|tfidf_features                                                                  |
+--------------------------------------------------------------------------------+
|(210917,[11,13,517],[14.031615435522635,9.475626921014051,6.827788748744692])
|(210917,[0,11,13,4555,20603],[1.1278235090076965,4.677205145174212,4.7378134605070255,8.994771533533466,11.18808985590262])
|(210917,[0],[0.2819558772519241])
|(210917,[110,517,676,852,1425],[5.613005028314541,6.827788748744692,7.0637891366811,7.271166595656634,7.696378466699633])
|(210917,[0,15,41,80,159,363,1970,2781],[0.8458676317557723,4.775868362380808,5.150135197913108,5.463587848441875,5.840564701458054,6.6522885519342045,8.000577970327878,8.357977289524651])|
+--------------------------------------------------------------------------------+
only showing top 5 rows
26/02/08 00:47:02 WARN DAGScheduler: Broadcasting large task binary with size 3.3 MiB
```

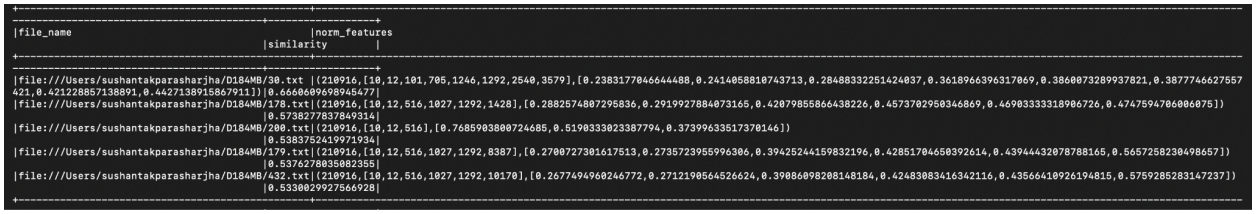
Using the trained IDF model to create TF-IDF feature vectors for every book.

```
...
... # Normalize
... normalizer = Normalizer(inputCol="tfidf_features", outputCol="norm_features")
... norm_df = normalizer.transform(tfidf_df)
...
... norm_df.select("norm_features").show(5, False)
...
```

```
+--------------------------------------------------------------------------------+
|norm_features                                                                   |
+--------------------------------------------------------------------------------+
|(210917,[11,13,517],[0.7685903800724685,0.5190333023387794,0.37399633517370146])
|(210917,[0,11,13,4555,20603],[0.07109214852090885,0.2948267704900404,0.29864720455692156,0.566983777751028,0.7052391968677033])
|(210917,[0],[1.0])
|(210917,[110,517,676,852,1425],[0.36222303658301513,0.4406164543309324,0.45584622460939667,0.46922887659885043,0.4966689972371513])
|(210917,[0,15,41,80,159,363,1970,2781],[0.04948970581313439,0.2794247130202582,0.30132217651305,0.3196615465026737,0.3417175666809214,0.3892096009584172,0.4680948120271521,0.4890053972071862])|
+--------------------------------------------------------------------------------+
only showing top 5 rows
26/02/08 00:47:02 WARN DAGScheduler: Broadcasting large task binary with size 3.3 MiB
26/02/08 00:47:02 WARN DAGScheduler: Broadcasting large task binary with size 3.3 MiB
```

Each book's TF-IDF vectors are normalised in order to get them ready for the cosine similarity calculation.

```
>>> from pyspark.sql.functions import desc
...
... # One vector per book
... book_vectors = norm_df.groupBy("file_name") \
...     .agg(first("norm_features").alias("norm_features"))
...
... # target vector
... target_vector = book_vectors \
...     .filter(col("file_name").contains("10.txt")) \
...     .select("norm_features") \
...     .head()[0]
...
... # cosine similarity
... cosine_udf = udf(lambda v: float(v.dot(target_vector)), DoubleType())
...
... similarity_df = book_vectors.withColumn(
...     "similarity",
...     cosine_udf(col("norm_features"))
... )
...
... # remove same book
... similarity_df \
...     .filter(~col("file_name").contains("10.txt")) \
...     .orderBy(desc("similarity")) \
...     .show(5, False)
```



Using normalised TF-IDF feature vectors, calculate the cosine similarity between the chosen book and every other book.

Identifying the top-5 books most similar to the selected book based on cosine similarity scores.

Each book was represented as a normalized TF-IDF vector capturing the importance of terms within the document. The cosine similarity between the selected book and all other books was computed using vector dot products. The books were then ranked according to similarity scores, and the top-5 most similar books were identified, providing a measure of textual similarity across the dataset.

Explain TF and IDF. Why is TF-IDF useful?

-> Term Frequency (TF) measures how often a word appears in a document, indicating the importance of that word within the specific document. Inverse Document Frequency (IDF) measures how rare a word is across the entire collection of documents; words that appear in many documents receive lower IDF values, while rare words receive higher values. TF-IDF combines these two measures by multiplying TF and IDF, giving higher weights to words that are frequent in a particular document but rare across the corpus. This helps highlight meaningful keywords while reducing the influence of very common words.

How is cosine similarity calculated? Why is it appropriate for comparing documents represented as TF-IDF vectors?

-> Cosine similarity is calculated as the dot product of two document vectors divided by the product of their vector magnitudes. It measures the cosine of the angle between the vectors rather than their absolute length. When documents are represented as TF-IDF vectors, cosine similarity effectively measures how similar their word distributions are, regardless of document size. This makes it suitable for comparing text documents because longer documents do not automatically receive higher similarity scores.

Discuss scalability challenges when calculating pairwise cosine similarity for many documents. How could Spark help address these challenges?

-> Calculating pairwise cosine similarity for a large number of documents requires comparing every document with every other document, which results in very high computational and memory costs as the dataset grows. This process becomes difficult to handle on a single machine. Apache Spark addresses these challenges by distributing both data storage and

computation across multiple nodes in a cluster, allowing similarity calculations to be processed in parallel. Spark's distributed DataFrame and ML libraries enable efficient large-scale vector processing, significantly reducing execution time for large document collections.

Question 12 - Author Influence Network

```
>>> from pyspark.sql.functions import *
...
... # Extract author + release date
... author_df = books_df.filter(
...     lower(col("text")).startswith("author:") |
...     lower(col("text")).startswith("release date:")
... ).groupBy("file_name").agg(
...     max(when(lower(col("text")).startswith("author:"), col("text"))).alias("author"),
...     max(when(lower(col("text")).startswith("release date:"), col("text"))).alias("release_date")
... )
...
... author_df.show(5, False)
...
```

```
+----------------------------------------------+---------------------------------+-----------------------------------+
|file_name                                     |author                           |release_date                       |
+----------------------------------------------+---------------------------------+-----------------------------------+
|file:///Users/sushantakparasharjha/D184MB/10.txt |NULL                          |Release Date: March 2, 2011 [EBook #10]|
|file:///Users/sushantakparasharjha/D184MB/101.txt|Author: Bruce Sterling        |Release Date: January, 1994        |
|file:///Users/sushantakparasharjha/D184MB/102.txt|Author: Mark Twain (Samuel Clemens)|Release Date: January, 1994        |
|file:///Users/sushantakparasharjha/D184MB/103.txt|Author: Jules Verne           |Release Date: May 15, 2008 [EBook #103]|
|file:///Users/sushantakparasharjha/D184MB/104.txt|Author: Franklin Delano Roosevelt  |Release Date: May 14, 2008 [EBook #104]|
+----------------------------------------------+---------------------------------+-----------------------------------+
only showing top 5 rows
```

Extracting author and release date metadata from each book for constructing the influence network.

```
...
... # Extract year
... author_year_df = author_df \
...     .withColumn("year_str", regexp_extract(col("release_date"), "(\\d{4})", 1)) \
...     .filter(col("year_str") != "") \
...     .withColumn("year", col("year_str").cast("int"))
...
... author_year_df.show(5, False)
...
```

```
+----------------------------------------------+---------------------------------+-----------------------------------+--------+----+
|file_name                                     |author                           |release_date                       |year_str|year|
+----------------------------------------------+---------------------------------+-----------------------------------+--------+----+
|file:///Users/sushantakparasharjha/D184MB/10.txt |NULL                          |Release Date: March 2, 2011 [EBook #10]|2011    |2011|
|file:///Users/sushantakparasharjha/D184MB/101.txt|Author: Bruce Sterling        |Release Date: January, 1994        |1994    |1994|
|file:///Users/sushantakparasharjha/D184MB/102.txt|Author: Mark Twain (Samuel Clemens)|Release Date: January, 1994        |1994    |1994|
|file:///Users/sushantakparasharjha/D184MB/103.txt|Author: Jules Verne           |Release Date: May 15, 2008 [EBook #103]|2008    |2008|
|file:///Users/sushantakparasharjha/D184MB/104.txt|Author: Franklin Delano Roosevelt  |Release Date: May 14, 2008 [EBook #104]|2008    |2008|
+----------------------------------------------+---------------------------------+-----------------------------------+--------+----+
only showing top 5 rows
```

Extracting the release year from the release date field for influence analysis.

```
... # Influence edges
... X = 5
...
... influence_edges = author_year_df.alias("a").join(
...     author_year_df.alias("b"),
...     (col("b.year") > col("a.year")) &
...     ((col("b.year") - col("a.year")) <= X) &
...     (col("a.author") != col("b.author"))
... ).select(
...     col("a.author").alias("author1"),
...     col("b.author").alias("author2")
... )
...
... influence_edges.show(10, False)
...
```

```
+----------------------+-------------------------------------+
|author1               |author2                              |
+----------------------+-------------------------------------+
|Author: Bruce Sterling|Author: Project Gutenberg            |
|Author: Bruce Sterling|Author: Robert Service               |
|Author: Bruce Sterling|Author: Henry James                  |
|Author: Bruce Sterling|Author: Unknown                      |
|Author: Bruce Sterling|Author: Andrew Barton 'Banjo' Paterson|
|Author: Bruce Sterling|Author: Lao-Tze                      |
|Author: Bruce Sterling|Author: Joseph Conrad                |
|Author: Bruce Sterling|Author: Joseph Conrad                |
|Author: Bruce Sterling|Author: G. K. Chesterton             |
|Author: Bruce Sterling|Author: Theodore Dreiser             |
+----------------------+-------------------------------------+
only showing top 10 rows
```

Construction of directed author influence edges based on publication year difference within a 5-year window.

```
...
... # Out-degree
... out_degree = influence_edges.groupBy("author1") \
...     .count() \
...     .withColumnRenamed("count","out_degree")
...
... out_degree.show(10, False)
...
...
... # In-degree
... in_degree = influence_edges.groupBy("author2") \
...     .count() \
...     .withColumnRenamed("count","in_degree")
...
... in_degree.show(10, False)
...
```

```
only showing top 10 rows
+--------------------------------------------+----------+
|author1                                     |out_degree|
+--------------------------------------------+----------+
|Author: Horatio Alger Jr.                   |177       |
|Author: Andrew Barton 'Banjo' Paterson      |119       |
|Author: Joanot Martorell and Marti Johan d'Galba|53        |
|Author: Virginia Woolf                      |177       |
|Author: Henry Lawson                        |13        |
|Author: John Bunyan                         |13        |
|Author: Michael Hart                        |53        |
|Author: Jules Verne                         |255       |
|Author: Various                             |325       |
|Author: Giuseppe Salza                      |53        |
+--------------------------------------------+----------+
only showing top 10 rows
+--------------------------------------------+---------+
|author2                                     |in_degree|
+--------------------------------------------+---------+
|Author: Horatio Alger Jr.                   |12       |
|Author: Andrew Barton 'Banjo' Paterson      |178      |
|Author: Joanot Martorell and Marti Johan d'Galba|56       |
|Author: Henry Lawson                        |66       |
|Author: Virginia Woolf                      |12       |
|Author: John Bunyan                         |66       |
|Author: Michael Hart                        |56       |
|Author: H. B. Irving                        |116      |
|Author: Jules Verne                         |130      |
|Author: Giuseppe Salza                      |56       |
+--------------------------------------------+---------+
only showing top 10 rows
```

Computation of in-degree and out-degree for each author in the influence network.

```
...
... # Top 5 most influential (OUT)
... out_degree.orderBy(desc("out_degree")).show(5, False)
...
...
... # Top 5 most influenced (IN)
... in_degree.orderBy(desc("in_degree")).show(5, False)
...
```

```
only showing top 10 rows
+-----------------------------+----------+
|author1                      |out_degree|
+-----------------------------+----------+
|Author: Thomas Hardy         |828       |
|Author: Robert Louis Stevenson|817       |
|Author: Edgar Rice Burroughs |720       |
|Author: Henry James          |613       |
|Author: Frances Hodgson Burnett|551       |
+-----------------------------+----------+
only showing top 5 rows
+-----------------------------+---------+
|author2                      |in_degree|
+-----------------------------+---------+
|Author: Robert Louis Stevenson|1911     |
|Author: Edgar Rice Burroughs |941      |
|Author: Arthur Conan Doyle   |615      |
|Author: L. Frank Baum        |552      |
|Author: Richard Harding Davis|512      |
+-----------------------------+---------+
only showing top 5 rows
```

Top 5 authors with the highest out-degree and in-degree showing the most influential authors and the most influenced authors based on the directed author influence network constructed using a 5-year publication window.

How was the influence network represented in Spark? What are the advantages and disadvantages of this representation?

-> The influence network was represented as a Spark DataFrame containing edges (author1, author2), where each row indicates that author1 potentially influenced author2 based on the release year condition.

Advantages

- DataFrames provide optimized query execution through Spark's Catalyst optimizer.
- Built-in aggregation functions allow easy computation of in-degree and out-degree.
- Scales efficiently for large datasets due to distributed processing.

Disadvantages

- DataFrame joins required for building the network can be computationally expensive for very large datasets.
- This representation does not directly support advanced graph algorithms unless converted to a graph framework such as GraphFrames.

How does the choice of the time window (X) affect the structure of the influence network?

-> The time window **X** determines how many connections are created between authors.
A small value of X results in fewer edges, producing a sparse network that captures only closely related publication periods.
A larger value of X creates more edges, producing a denser network where many authors appear connected, which may introduce less meaningful influence relationships.

What are the limitations of this simplified definition of influence?

-> Actual intellectual influence between authors is not captured by this method, which assumes influence based only on publication years. Citations, literary style, historical context, and teamwork are all necessary for true influence, but none of these are taken into account here. As a result, the network only offers a rough picture of possible influence.

How well would this approach scale to a much larger dataset? What optimizations could be considered?

-> Spark spreads computation across several nodes, allowing the method to scale to large datasets. However, as the number of authors rises, the self-join that is used to generate edges may become more costly.

Possible optimizations include:

- Partitioning the dataset by publication year to reduce join cost.
- Filtering invalid or missing metadata before network construction.
- Using broadcast joins when one dataset is small.

- Employing graph-processing frameworks such as GraphFrames for efficient large-scale network analysis.